

Membres : Ilias Hormatallah ; Nicolas Manent ; Raphael Michel-Espinosa ; Manon Gay

Rapport Fondement IA

1. Introduction

Notre projet a débuté à la suite du CM de présentation. Lors de ce CM, Il a été précisé que d'autres langages de programmation autres que Python étaient possibles, notamment à la suite d'une question sur Rust.

Ainsi, à l'aide de cette déclaration, nous avons choisi d'implémenter ce projet en C++ pour plusieurs raisons :

- La plupart des membres du groupe sont plus performants sur C++
- L'exemple de Rust a été donné, sur lequel nous avons hésité car cela nous aurait permis de prendre en main un nouveau langage etc. Mais, d'après nos recherches, Rust n'est pas natif à Google Colab et risquerait de poser des complications le jour de la Battle IA
- Les performances : C++ étant un langage de plus bas niveau que Python, on calculera plus rapidement que sur Python, ce qui permettra d'explorer davantage de nœuds dans le même budget de temps

Ce qui est particulièrement important pour Minimax/Alpha-Beta, où de très nombreuses opérations similaires sont répétées. Donc, la réelle contrainte de notre projet sera d'optimiser la complexité.

Benchmark :

Pour déterminer notre progression, nous avons codé en Python un benchmark qui testera notre modèle de base :

```
=====
  ULTIMATE TIC TAC TOE v2.0
=====
1. Human vs AI
2. AI vs AI (verbose)
3. Self-play tournament
4. Quit
> 4

ninel@LAPTOP-KBVT190E UCRT64 /c/Users/ninel/Documents/R
# ./utt_champion.exe cli "0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 -1 1" 300
echo $?
5 5
0
```

On aura le choix de le challenger nous-mêmes, où avec des « IA » prédéfinies, comme un adversaire qui ne joue que de manière aléatoire, un Minimax basique avec une profondeur de 2 ou 3 etc.

La limite ici, c'est que notre modèle doit être compatible CLI, ce qui nous rajoute du code, voire de la performance sur un point qui, en théorie, ne sera pas utile lors du tournoi. Mais, afin d'évaluer les performances de notre modèle, nous avons jugé que cela restait nécessaire.

2. Représentation de l'état

Afin de se conformer au sujet, on devait « traduire » le travail effectué lors des travaux pratiques en Python en C++. Pour cela, on doit savoir comment représenter notre plateau.

En TP sur Python, l'approche était plus visuelle, même si coûteuse, cela nous permettait de visualiser facilement l'état du plateau.

Nous, on stockera les plateaux en bitboards, ainsi, cela ne nous prendra que 9 bits pour représenter les 9 cases d'un morpion basique, où l'on y stockera un entier binaire pour chaque bit.

Par exemple : si X a la case 0, 4, 8

Bit : 8 7 6 5 4 3 2 1 0

1 0 0 0 1 0 0 0 1

Donc, en binaire : 100010001

Grâce à ça, on peut connaître les codes binaires des lignes / diagonales gagnantes, puis, on les traduit en hexadécimal par souci de lisibilité.

À la suite de cela, on attribue des scores aux cases vides, car intuitivement, on sait qu'au Morpion (que ce soit le Ultimate ou le classique) la meilleure case c'est le centre, puis les diagonales, et enfin les bords, notre modèle doit donc prendre ça en compte. Par ailleurs, s'il voit une case gagnante, il doit être fortement incité à la prendre, pour cela, on met un score très grand pour qu'il la prenne prioritairement.

Concernant la détection des victoires, au lieu de vérifier à chaque fois si une condition de victoire est remplie en regardant les cases, afin de limiter la complexité, on ne le fait qu'une fois : $IS_WIN[2^9 \text{ (9 bits)}]$, qui renverra un booléen selon le résultat.

THREATS s'inscrit dans la même logique, mais appliqué aux menaces.

Popcnt lui, servira à compter le nombre de bits en faisant un peu de bit-hacking : on regroupe les bits petit à petit, on additionne localement, et à la fin on récupère le total.

Pour la clarté de ce rapport, on parlera de Minimax (et Alpha-Beta) à la fin, étant donné qu'à chaque fois que l'on rajoute / optimise une technique, on devra probablement repasser par l'Alpha-Beta, et donc, celui-ci sera amené à changer continuellement au fur et à mesure de l'avancement du projet, risquant de perturber la chronologie de ce rapport.

Zobrist hashing :

L'objectif est d'attribuer à chaque position du jeu une signature numérique quasi unique, calculée très rapidement.

En ayant connaissance que dans ce jeu, l'algorithme va revisiter plusieurs fois la même position, et si à chaque fois on recalcule tout depuis zéro, on perd énormément de temps. Donc, on veut pouvoir faire comprendre au modèle "ah, cette position, je l'ai déjà vue".

Pour cela, on lui associe un identifiant unique, le « Hash », qui sera concrètement un nombre calculé à partir de nombres aléatoires, avec une seed fixe pour la reproductibilité. Ce choix vise à optimiser la complexité, comparer des tuples décrivant les positions déjà explorées, est en réalité très coûteux.

State :

C'est la représentation complète d'une position de jeu à un instant donné. Pour résumer, State répond à : « où sont les X et les O, quels sous-plateaux sont gagnés, qui doit jouer, où il a le droit de jouer, et quel est le hash de la position ? »

Pour cela, on définit les encodeurs & décodeurs d'un plateau 3x3, avec mv_sub et mv_cell et Move. Qu'ensuite on élargit en 9x9 avec mv_col et mv_row

Donc la structure State représente entièrement une position de jeu. Elle contient les bitboards bx et bo, qui stockent respectivement les cases occupées par X et O dans chacun des 9 sous-plateaux. Les variables meta_x et meta_o indiquent quels sous-plateaux ont été remportés par chaque joueur, tandis que meta_done signale les sous-plateaux terminés, qu'ils soient gagnés ou nuls.

La variable constraint mémorise le sous-plateau imposé au prochain joueur, (c'est le jeu Ultimate Tic-Tac-Toe qui impose ça), et current indique le joueur dont c'est le tour. Enfin, hash contient la signature de Zobrist de la position, utilisée pour la table de transposition. Autour de cette structure, les fonctions gen_moves, do_move, un_move et terminal permettent respectivement de générer les coups légaux, d'appliquer un coup, d'annuler un coup et de détecter une fin de partie.

3. Optimisation / Recherches

Evaluation :

L'objectif est le suivant : si la recherche s'arrête à ce stade, est-ce que cette position a l'air favorable pour X ou pour O ?

Le principe est le suivant :

- score positif → avantage X
- score négatif → avantage O
- score grand en valeur absolue → gros avantage

La fonction evaluate se limite à déterminer si la partie est terminée ou non, et si oui, qui est le vainqueur. Dans le cas où on a un match nul, elle gardera quand même une nuance en regardant qui a le plus de sous-plateaux gagnés, et en déduit un score.

Nous développons en détail la **partie heuristique eval_leaf** ici, étant le sujet principal du projet.

L'objectif de l'heuristique est d'estimer la valeur d'une position lorsque la recherche alpha-beta atteint sa profondeur limite. C'est la ligne « if (depth <= 0) return eval_leaf(st); » dans l'alpha-beta, sans elle, le modèle devrait explorer chaque partie jusqu'à la fin, ce qui est impossible étant donné la taille de l'arbre

Soit sc le score calculé. Un score positif indique un avantage pour X, un score négatif un avantage pour O. L'heuristique additionne plusieurs composantes :

Pour chaque sous-grille encore en jeu, on consulte une table (précalculée) EVAL_SUB[512x512] indexée par les bitboards de X et O. Cette table est construite quand le programme est lancé : pour chaque configuration possible de X et O dans un morpion 3x3, on calcule un score prenant en compte les menaces (deux pions alignés sans blocage adverse), les lignes ouvertes (un pion aligné sans blocage), et les cases intuitivement meilleurs (centre puis coins puis bords).

L'intérêt de cette table est surtout la complexité : au lieu de recalculer ces critères à chaque appel (boucle sur 8 patterns de victoire x 9 sous-grilles), un simple accès suffit, soit $O(1)$ par sous-grille. Étant donné que eval_leaf est appelée des millions de fois par seconde, cela fait une énorme différence.

Chaque sous-grille est ensuite influencée par sa position sur le méta-board avec SUB_W[i] : le centre vaut 5, les coins 4, les bords 3. Intuitivement, contrôler le sous-plateau au milieu est aussi plus avantageux

Au-delà du score, c'est également ce mécanisme qui guide la recherche : une ligne comportant déjà un X est plus prometteuse, donc, on donne un bonus (ici 400 pour un X, 2500 pour deux X). À l'inverse, des malus sont appliqués en présence de O.

Si un joueur possède au moins deux lignes avec chacune deux sous-grilles gagnées, il dispose d'une double menace : l'adversaire ne peut bloquer qu'une seule ligne à la fois, donc la victoire est quasi assurée. Ce cas est récompensé par un bonus supplémentaire de 3000 points.

Concernant la contrainte : il s'agit de faire comprendre au modèle que parfois, la contrainte peut être soit favorable, soit piègeuse.

À cette fin, on regarde les menaces du joueur par rapport à celles de l'adversaire, qu'on compare à l'aide d'une soustraction, si le résultat est positif, le joueur a plus de menaces que l'adversaire, et donc, la contrainte est favorable. Sinon, on a l'inverse.

Lorsque le joueur a le choix libre (aucune contrainte), un léger bonus de tempo est accordé, car pouvoir choisir sa sous-grille est un avantage.

Table de transposition (TT) :

Le complément au hashing de Zobrist, il s'agit d'un répertoire de toutes nos recherches, il permet de mémoriser les positions déjà évaluées durant la recherche. Chaque entrée contient le hash Zobrist de la position, le score associé, la profondeur d'analyse, le meilleur

coup trouvé ainsi qu'un indicateur précisant si le score est exact ou s'il s'agit d'une borne supérieure ou inférieure.

Lorsqu'une position réapparaît dans l'arbre de recherche, le moteur peut réutiliser directement ces informations, ce qui évite des recalculs coûteux et améliore l'efficacité de l'algorithme alpha-bêta.

Sans la TT, le moteur devrait recalculer intégralement les positions déjà évaluées.

Amélioration du benchmark

Cependant, plus le modèle devenait technique, plus le benchmark devenait inutile, tous les modèles le passaient avec juste une différence de vitesse qui était constatable. Mais on ne pouvait pas voir à quel point il était performant. Ainsi, on développait sans réellement savoir si, de l'autre côté, nous ne réduisions pas involontairement les performances.

Pour y remédier, un programme Python aux règles du tournoi a été développé, permettant de confronter deux modèles compilés en C++ sur une partie d'Ultimate Tic-Tac-Toe. Les temps de réponse sont chronométrés, on regarde qui a gagné, puis à la fin, les points sont calculés suivant le barème du projet.

```
R40a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_v3.exe WIN [57mv 4-5]
R40b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 5-1]
R41a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [55mv 4-3]
R41b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 6-3]
R42a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [59mv 6-3]
R42b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [57mv 5-3]
R43a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [59mv 6-3]
R43b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 5-1]
R44a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [57mv 3-1]
R44b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 5-1]
R45a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN (fast) [57mv 3-1]
R45b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 6-3]
R46a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN (fast) [59mv 6-3]
R46b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [57mv 5-3]
R47a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [56mv 5-3]
R47b uttt_v3.exe(X) vs uttt_alt.exe(O): DRAW [60mv 4-4]
R48a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN (fast) [59mv 6-3]
R48b uttt_v3.exe(X) vs uttt_alt.exe(O): DRAW [55mv 4-4]
R49a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_alt.exe WIN [51mv 5-3]
R49b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 5-1]
R50a uttt_alt.exe(X) vs uttt_v3.exe(O): uttt_v3.exe WIN (fast) [61mv 4-5]
R50b uttt_v3.exe(X) vs uttt_alt.exe(O): uttt_v3.exe WIN [59mv 5-1]

=====
uttt_alt.exe: 216 pts (46 victoires)
uttt_v3.exe: 172 pts (50 victoires)
Nuls: 4
```

Move Ordering :

Cette partie sert surtout à guider la recherche de l'Alpha-Beta. L'Alpha-Beta parcourant l'arbre des possibilités, l'enjeu est de l'orienter en priorité vers les coups les plus prometteurs, afin qu'il puisse s'arrêter plus tôt.

Tel est précisément l'objectif du Move Ordering : un score est attribué à chaque coup possible, et ces derniers sont triés du plus prometteur au moins prometteur. Nous avons cependant fait attention à ce que la TT ait la priorité, si elle a déjà indiqué qu'un coup était meilleur dans cette position, il a de fortes chances d'être le bon.

Il est ainsi possible de donner de l'importance aux coups qui bloquent l'adversaire : si l'adversaire a déjà deux cases sur une ligne par exemple, alors le Move Ordering attribuera un score qui fera en sorte que le modèle le bloque. Créer des killers moves aussi, soit des coups qui ont déjà provoqué une coupure à cette profondeur dans une autre branche (on fait ça car un coup déjà très fort auparavant a de fortes chances d'être encore utile).

Par ailleurs, dans le contexte de l'Ultimate Tic-Tac-Toe, il y a aussi de la stratégie pour ne pas envoyer l'adversaire vers un sous-plateau qui l'avantagerait. Ainsi, on baisse le score si le plateau est dangereux pour nous. Cela poussera le modèle à aller plus loin que localement.

Root Search :

La partie Root Search, c'est la recherche faite au nœud racine, c'est-à-dire la position actuelle depuis laquelle le moteur doit concrètement choisir un coup à jouer. C'est différent de l'Alpha-Beta classique, car l'Alpha-Beta cherche surtout un score. Ici, c'est le meilleur coup concret à retourner.

Pour cela, d'abord, on regarde en priorité la TT afin de récupérer un éventuel tt_move, qui servira à prioriser l'exploration des coups.

Ensuite, on génère tous les coups légaux possibles (81 en théorie), si aucun n'est disponible, out est fixé à -1 et la fonction retourne 0.

Le score heuristique est ensuite calculé pour chaque coup, puis les coups sont triés via le Move Ordering, ici focalisé sur la racine, de sorte que le meilleur coup soit examiné en premier, et que la borne alpha monte plus rapidement.

Puis, la maximisation ou minimisation s'effectue selon le joueur actif, et on initialise bm (best move) avant de faire une boucle sur les coups. Où pour chaque coup :

- On l'applique
- On appelle alphabeta sur la position fille
- On annule le coup
- On compare le score obtenu avec le meilleur courant

Un objet Undo est créé pour pouvoir restaurer l'état après avoir simulé le coup. Sans cela, on ne pourrait pas tester plusieurs coups proprement.

Cette approche, apply/undo, évite de copier l'intégralité de l'état à chaque nœud exploré (18 bitboards + méta-données, donc un coût en $O(81)$). Au lieu de cela, seuls les champs modifiés sont sauvegardés dans l'objet Undo, et restaurés après exploration. Le coût passe à $O(1)$ par nœud, ce qui fait une énorme différence sachant que cette opération est répétée des millions de fois.

Le premier coup, lui, est recherché dans la fenêtre complète [alpha ; beta]. C'est une optimisation due au PVS, qui suggère que le premier coup après tri sera certainement le meilleur.

Pour les coups suivants, une recherche est d'abord effectuée dans une fenêtre très étroite, afin de déterminer si le coup dépasse le meilleur résultat actuel.

S'il a trouvé quelque chose d'intéressant, une recherche complète est lancée. Ce filtre permet d'économiser des ressources, et de ne déclencher une recherche complète que si c'est nécessaire. C'est le principe du PVS à la racine.

À la fin, le meilleur coup est retourné, que ce soit dû à la contrainte de temps, ou de la découverte du coup optimal.

Serialization :

La Serialization permet de convertir une position entre sa représentation interne (State) et une représentation en texte qui sera exploitable en dehors, c'est pour cela que les fonctions suivantes existent :

- Compute Hash : Cette fonction reconstruit le hash complet à partir de l'état.

Elle recalcule le hash pour chaque case : si X est présent, XOR avec la clé Zobrist de X, si O est présent, XOR avec la clé Zobrist de O. Puis, ajoute la clé du joueur actif, ainsi que la clé associée à la contrainte, afin d'obtenir le hash final.

- Parse State : C'est la fonction qui lit une position depuis du texte.

Une liste de taille 23 est sécurisée (18 valeurs pour bx et bo, 3 pour les meta, 1 pour constraint, 1 pour current), si les 23 entiers ne peuvent pas être lus, la fonction retourne false, car la position étant considérée invalide. Dans le cas contraire, on remplit les bitboards de variables (avec un masquage binaire pour être sûr de n'avoir que 9 bits)

- Serialize : Transforme State en chaîne de caractères

L'inverse de Parse State : elle transforme la structure State en texte. Pour cela, des boucles sont effectuées sur bx et bo, suivies des autres champs, et on aura les 23 exactement dans le bon ordre.

Alpha-Beta :

Étant donné que cette partie est la base de notre algorithme de recherche qu'on a notamment déjà vue en TD/CM, on fera un résumé rapide de ce que c'est, mais on verra surtout ce qu'on y a ajouté.

L'algorithme alphabeta est une optimisation du minimax. Comme minimax, il explore récursivement les coups possibles en supposant qu'un joueur cherche à maximiser le score tandis que l'autre cherche à le minimiser. Sa différence principale est l'utilisation de deux bornes, alpha et beta qui réduisent fortement le nombre de positions explorées.

Avant tout, notre alpha-bêta doit être interruptible en raison de la contrainte de temps, pour cela, il vérifiera périodiquement si le temps est écoulé.

Notre test terminal permettra de renvoyer le score, avec un +/- dépendamment de si le modèle gagne ou non. On préfère les victoires rapides, et les défaites plus tardives si elles sont inévitables. Si on a un nul, on nuance dépendamment du nombre de sous plateaux gagnés.

Une feuille de recherche est définie : si elle atteint la profondeur limite, la recherche s'arrête et l'évaluation heuristique `eval_leaf` est utilisé.

La TT est ensuite consultée, avec deux chemins distincts en fonction de si on a déjà vu ou non la position : si oui, on guide de manière importante la recherche, économisant énormément en performance. Au mieux, on renvoie directement le score, au pire une borne. Tout en gardant le `tt_move` pour améliorer le `move_ordering`.

Un LMR (Late Move Reduction) est appliqué :

Les coups non décisifs, situés loin dans la liste, et ne s'avérant pas localement gagnants sont supposés moins susceptibles d'être les meilleurs. Donc, on cherche d'abord moins profondément et si le coup paraît finalement intéressant, on relance une recherche complète

Cette hypothèse repose sur le constat que les derniers coups après tri sont souvent médiocres. Donc, chercher à pleine profondeur chacun d'eux coûterait en temps pour peu de gain.

Enfin, le PVS est appliqué à nouveau (cf Root Searching).

4. Tableau récapitulatif

Le tableau suivant résume ce les résultats obtenus à chaque nouvelle optimisation afin de marquer l'avancement du projet. Nous en profiterons aussi pour mettre ce qui n'a pas fonctionné.

Version	Victoires	Défaites	Nuls	Prof. moy.	Score
Alpha-Beta simple	4	24	2	~4	16
+ Table de transposition (Zobrist)	12	16	2	~6	31
+ Move ordering (killers, history)	15	12	3	~7	43
+ Heuristique pré-calculée (EVAL_SUB)	20	9	1	~8	59
+ Apply/Undo + eval_leaf	23	6	1	~9	67
+ LMR / PVS	26	3	1	~11	77

Stats mesurées sur 30 parties contre un adversaire C++ de référence, 1000ms/coup et un plateau initial vide. On considère généralement qu'à partir d'environ $n = 30$, l'échantillon est suffisamment grand pour utiliser certaines approximations statistiques, mais ils doivent être interprétés avec prudence : un échantillon de 30 parties reste limité.

Afin de compléter ces résultats, nous pouvons calculer un intervalle de confiance indicatif à 95 % sur le taux de victoire, par exemple avec l'intervalle de Wilson.

$$IC_{Wilson} = \frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}$$

Avec $z = 1.96$ pour un intervalle à 95 %.

Nous construisons un intervalle de confiance sur notre modèle le plus performant, soit la version LMR/PVS qui obtient 26 victoires sur 30, soit un taux de victoire observé de 86,7 %, avec un intervalle de confiance d'environ **[70,3 % ; 94,7 %]**. Les nuls étant pénalisés par le barème, ils sont assimilés à des défaites dans le calcul, soit $\hat{p} = 26/30$.

Modifications qui n'ont pas fonctionné

Modification testée	Résultat	Raison de l'échec	Leçon retenue
Comptage de pièces brut	Jeu sans stratégie	5 pions éparpillés valent autant que 3 alignés, aucune notion de menace	Ce sont les menaces qui comptent, pas le nombre de pièces.
Tuning heuristique (6 variantes)	Bruit statistique	Bugs TT rendaient les mesures invalides	Corriger les bugs avant d'optimiser
Meta-heavy (fork 1500, threat 600)	8-16 (perd)	Poids trop agressifs, déséquilibre l'évaluation	Plus agressif ≠ meilleur
Combo strategic + contrainte forte	7-12 (perd)	Deux bonnes idées combinées peuvent dégrader	Tester chaque changement isolément
Malus pour les sous-grilles nulles	6-14 (perd)	Un nul peut être stratégiquement souhaitable pour bloquer l'adversaire	Un résultat "neutre" localement peut être un gain globalement
Free choice loop	39-24 vs 51-21	Boucle ralentit l'eval : perte du bonus fast	Coût en performance > gain en qualité
Bonus fixe pour le centre de chaque sous-grille	Négatif en fin de partie	La valeur d'une case dépend du contexte (menaces autour), pas juste de sa position	Un bonus statique ne remplace pas une évaluation contextuelle
Tie-breaking aléatoire	(8,7) au lieu de (5,5)	Coups aléatoires en ouverture	Utiliser le PV de la profondeur N-1

5. Les choix d'optimisations

Maintenant que l'ensemble des détails techniques est établi, il convient d'optimiser le choix du temps idéal. Un phénomène remarquable a été mis en évidence : **search pathology**

```
$ python tournament.py ./uttt_PVS.exe ./uttt_PVS_65.exe --rounds 10 --time 10000

TOURNAMENT: uttt_PVS.exe vs uttt_PVS_65.exe
10 rounds | 10000ms/coup

R1a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R1b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [45mv 3-3]
  temps a: X=132480.7ms total (5760.0ms/coup, 23 coups) | O=147396.3ms total (6699.8ms/coup, 22 coups)
  temps b: X=144967.4ms total (6302.9ms/coup, 23 coups) | O=108997.7ms total (4954.4ms/coup, 22 coups)
R2a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R2b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): DRAW [57mv 4-4]
  temps a: X=136104.5ms total (5917.6ms/coup, 23 coups) | O=147219.1ms total (6691.8ms/coup, 22 coups)
  temps b: X=188683.1ms total (6506.3ms/coup, 29 coups) | O=145694.3ms total (5203.4ms/coup, 28 coups)
R3a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R3b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [59mv 6-3]
  temps a: X=130119.0ms total (5657.3ms/coup, 23 coups) | O=149898.3ms total (6813.6ms/coup, 22 coups)
  temps b: X=184401.2ms total (6146.7ms/coup, 30 coups) | O=136498.8ms total (4706.9ms/coup, 29 coups)
R4a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): DRAW [57mv 4-4]
R4b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): DRAW [57mv 4-4]
  temps a: X=156401.2ms total (5393.1ms/coup, 29 coups) | O=185531.2ms total (6626.1ms/coup, 28 coups)
  temps b: X=193470.7ms total (6671.4ms/coup, 29 coups) | O=140351.2ms total (5012.5ms/coup, 28 coups)
R5a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R5b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [45mv 3-3]
  temps a: X=135043.3ms total (5871.4ms/coup, 23 coups) | O=148721.7ms total (6760.1ms/coup, 22 coups)
  temps b: X=139304.4ms total (6056.7ms/coup, 23 coups) | O=115697.5ms total (5259.0ms/coup, 22 coups)
R6a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R6b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [45mv 3-3]
  temps a: X=134646.7ms total (5854.2ms/coup, 23 coups) | O=146764.0ms total (6671.1ms/coup, 22 coups)
  temps b: X=146909.4ms total (6387.4ms/coup, 23 coups) | O=114472.8ms total (5203.3ms/coup, 22 coups)
R7a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R7b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 4-2]
  temps a: X=128365.5ms total (5581.1ms/coup, 23 coups) | O=147766.7ms total (6716.7ms/coup, 22 coups)
  temps b: X=192737.4ms total (6023.0ms/coup, 32 coups) | O=152161.1ms total (4908.4ms/coup, 31 coups)
R8a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R8b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS.exe WIN (fast) [60mv 2-6]
  temps a: X=127269.6ms total (5533.5ms/coup, 23 coups) | O=147960.7ms total (6725.5ms/coup, 22 coups)
  temps b: X=195653.1ms total (6521.8ms/coup, 30 coups) | O=140437.1ms total (4681.2ms/coup, 30 coups)
R9a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R9b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [45mv 3-3]
  temps a: X=136353.0ms total (5928.4ms/coup, 23 coups) | O=148132.7ms total (6733.3ms/coup, 22 coups)
  temps b: X=143206.5ms total (6226.4ms/coup, 23 coups) | O=116068.8ms total (5275.9ms/coup, 22 coups)
R10a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [45mv 3-3]
R10b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [45mv 3-3]
  temps a: X=132785.6ms total (5773.3ms/coup, 23 coups) | O=151407.0ms total (6882.1ms/coup, 22 coups)
  temps b: X=142792.6ms total (6208.4ms/coup, 23 coups) | O=113210.7ms total (5145.9ms/coup, 22 coups)

=====
uttt_PVS.exe: 50 pts (10 victoires)
uttt_PVS_65.exe: 21 pts (7 victoires)
Nuls: 3
=====
>>> uttt_PVS.exe GAGNE LE TOURNOI <<<
```

Bien que logiquement, on puisse supposer que davantage de temps de calcul améliore les résultats, les résultats du tournoi démontrent le contraire. En écartant l'aspect score, à 10 000 ms par coup, la version a 65% de limite de temps produit davantage de défaites que celle à 40% d'après les résultats ci-dessus. Ce résultat a été confirmé par de nombreux tests (il ne s'agit donc pas de bruit statistique), ce phénomène est appelé le **search pathology**.

L'explication la plus probable est que notre moteur utilise du LMR, PVS, du move ordering etc... Des optimisations impliquant que les premières recherches soient probablement les meilleures. Avec un temps plus long, le moteur explore davantage de positions nouvelles, mais les évalue donc avec une heuristique imparfaite.

Il convient donc de tester plusieurs configurations pour identifier le temps idéal, par exemple ci-dessous à 1000ms :

```
$ python tournament.py ./utt_PVS.exe ./utt_PVS_65.exe --rounds 10 --time 1000

TOURNAMENT: uttt_PVS.exe vs uttt_PVS_65.exe
10 rounds | 1000ms/coup

R1a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): DRAW [66mv 4-4]
R1b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): DRAW [61mv 4-4]
  temps a: X=17152.3ms total (519.8ms/coup, 33 coups) | O=22330.6ms total (676.7ms/coup, 33 coups)
  temps b: X=21443.0ms total (691.7ms/coup, 31 coups) | O=14873.4ms total (495.8ms/coup, 30 coups)
R2a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS.exe WIN (fast) [65mv 6-2]
R2b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=15628.4ms total (473.6ms/coup, 33 coups) | O=21422.5ms total (669.5ms/coup, 32 coups)
  temps b: X=22332.2ms total (697.9ms/coup, 32 coups) | O=13928.5ms total (449.3ms/coup, 31 coups)
R3a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R3b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [65mv 6-2]
  temps a: X=13889.1ms total (514.4ms/coup, 27 coups) | O=17622.8ms total (652.7ms/coup, 27 coups)
  temps b: X=22818.2ms total (691.5ms/coup, 33 coups) | O=14519.0ms total (453.7ms/coup, 32 coups)
R4a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R4b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=14219.5ms total (527.4ms/coup, 27 coups) | O=17602.9ms total (652.0ms/coup, 27 coups)
  temps b: X=21823.6ms total (682.0ms/coup, 32 coups) | O=13661.4ms total (440.7ms/coup, 31 coups)
R5a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [72mv 2-5]
R5b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=16954.2ms total (470.9ms/coup, 36 coups) | O=23646.4ms total (656.8ms/coup, 36 coups)
  temps b: X=21543.2ms total (673.2ms/coup, 32 coups) | O=13610.0ms total (439.0ms/coup, 31 coups)
R6a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R6b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=12915.9ms total (478.4ms/coup, 27 coups) | O=17058.4ms total (631.8ms/coup, 27 coups)
  temps b: X=21574.3ms total (674.2ms/coup, 32 coups) | O=13255.5ms total (427.6ms/coup, 31 coups)
R7a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [53mv 3-5]
R7b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [65mv 6-2]
  temps a: X=15572.0ms total (576.7ms/coup, 27 coups) | O=16695.9ms total (642.1ms/coup, 26 coups)
  temps b: X=22455.6ms total (680.5ms/coup, 33 coups) | O=14460.1ms total (451.9ms/coup, 32 coups)
R8a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R8b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [65mv 6-2]
  temps a: X=14614.1ms total (541.3ms/coup, 27 coups) | O=17685.8ms total (655.0ms/coup, 27 coups)
  temps b: X=22740.4ms total (689.1ms/coup, 33 coups) | O=14403.0ms total (450.1ms/coup, 32 coups)
R9a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R9b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=14255.6ms total (528.0ms/coup, 27 coups) | O=17064.9ms total (632.0ms/coup, 27 coups)
  temps b: X=21901.2ms total (684.4ms/coup, 32 coups) | O=13370.8ms total (431.3ms/coup, 31 coups)
R10a uttt_PVS.exe(X) vs uttt_PVS_65.exe(O): uttt_PVS_65.exe WIN [54mv 2-6]
R10b uttt_PVS_65.exe(X) vs uttt_PVS.exe(O): uttt_PVS_65.exe WIN [63mv 6-2]
  temps a: X=13631.2ms total (504.9ms/coup, 27 coups) | O=17437.9ms total (645.8ms/coup, 27 coups)
  temps b: X=21763.5ms total (680.1ms/coup, 32 coups) | O=13437.9ms total (433.5ms/coup, 31 coups)

=====
utt_PVS.exe: 23 pts (1 victoires)
utt_PVS_65.exe: 51 pts (17 victoires)
Nuls: 2
>>> uttt_PVS_65.exe GAGNE LE TOURNOI <<<=====
```

Sans le bonus de rapidité, le modèle à 65 % l’emporte nettement sur celui à 40 % dans un temps limité d’une seconde par coup. Il convient donc de trouver l’équilibre optimal entre qualité de recherche et temps accordé.

Afin de l’évaluer face à des modèles indépendants, nous avons téléchargé des projets similaires adoptant d’autres approches.

Approche Monte-Carlo Search (Python) :

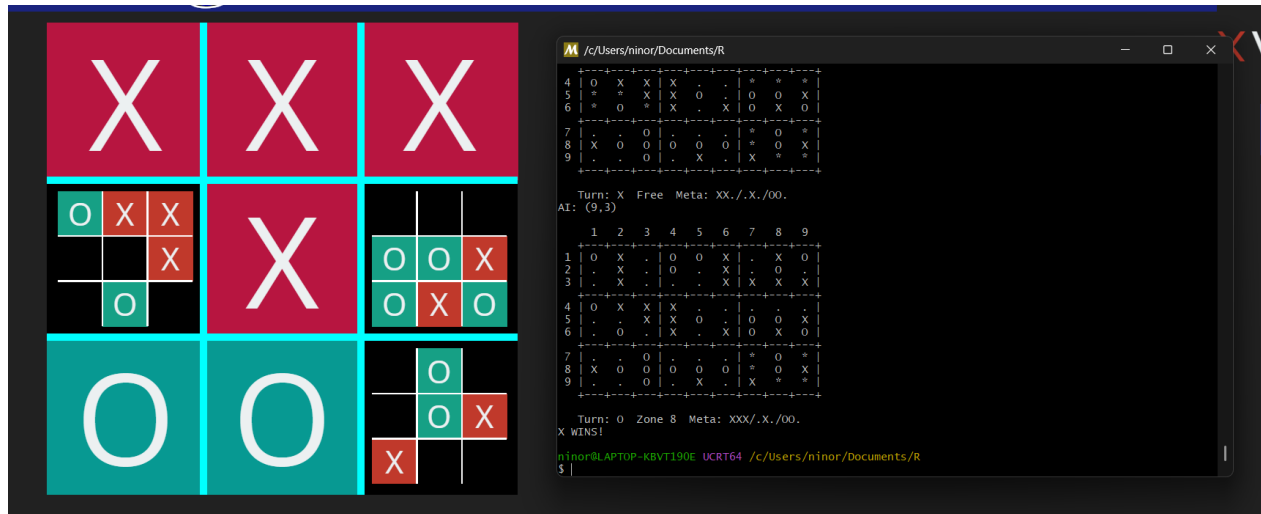
```
/c/Users/ninor/Documents/R
8 | . X 0 0 0 * 0 | . 0 . |
9 | . 0 | * X * 0 | . . |
-----
Turn: 0 Free Meta: 0.X/.X./0.0
AI: (5,8)
  1 2 3 4 5 6 7 8 9
1 | . . X * * 0 | X 0 0 |
2 | 0 0 0 | X X | X 0 X |
3 | . X . | X * 0 | . 0 X |
-----
4 | X * X | . 0 | X 0 0 | X |
5 | 0 X X | . . | X X 0 | |
6 | * 0 * | . 0 | X | X * X |
-----
7 | . . 0 | X X | . | X X 0 |
8 | . X 0 0 | 0 | . 0 | . |
9 | . 0 | . X . | 0 . . |
-----
Turn: X Free Meta: 0.X/.X./000
WINS!

/c/Users/ninor/Documents/R/MonteCarloTreeSearch-master
0 X X | . X X 0 0
- 0 | - 0 X | X X
-----
- 0 | X X | X X 0
- X 0 | 0 0 | 0 0
- 0 | - X | 0 0
-----
It is P2's turn now.
Enter 2 space separated integers of the form: [0-8] [0-8], corresponding to a valid move of course!
7 4
Move played by p2: [7, 4]
  0 0 0 | X X | 0 X X
- X | X 0 | 0 X
-----
X X | - 0 X | 0 X
0 X X | - X | X 0 0
- 0 | - 0 X | X X
-----
- 0 | X X | X X 0
- X 0 | 0 0 0 | 0
- 0 | - X | 0 0
-----
P2 won!

/c/Users/ninor/Documents/R/MonteCarloTreeSearch-master
```

Le modèle téléchargé étant en Python (incompatible avec le code du tournoi, d'où la nécessité de deux terminaux. Malgré des résultats relativement proches, le moteur développé s'est imposé.

Java/React :



Le moteur l'emporte sur le modèle le plus performant du site, ce qui marque la performance de notre moteur.

Conclusion :

Ce projet nous a permis de développer un moteur d'IA pour l'Ultimate Tic-Tac-Toe qui dépasse le cadre du simple Minimax vu en TP. Le choix du C++ s'est avéré déterminant : à temps égal, notre moteur atteint une profondeur de recherche d'environ 11, là où une implémentation Python équivalente plafonne à 4-5.

Au-delà de la performance brute du langage, c'est surtout la démarche itérative qui a été formatrice. Chaque optimisation (table de transposition, move ordering, heuristique pré-calculée, apply/undo, LMR, PVS) a été mesurée individuellement via un benchmark, et certaines tentatives ont échoué, parfois parce qu'elles dégradaient la vitesse plus qu'elles n'amélioraient la qualité de jeu, parfois parce que des bugs faussaient les mesures.

Enfin, le compromis entre qualité de recherche et rapidité, imposé par le barème du tournoi, nous a poussés à réfléchir au-delà de l'algorithme pur : le choix du seuil de temps, l'adaptation à l'adversaire probable, et le phénomène de search pathology observé expérimentalement montrent que l'optimisation d'une IA de jeu ne se limite pas à chercher plus profond, mais à chercher mieux.

Je vous remercie d'avoir lu.